Streamlining symbol files in Oberon

Andreas Pirklbauer

19.9.2025

Overview

This technical note presents a simplification and improvement of the handling of import and export[1] for the *Project Oberon 2013* system, which is a reimplementation of the original *Oberon* operating system on an FPGA development board in 2013, as published at *www.projectoberon.com.* If you use the *Extended Oberon*[2] system, the improvements presented in this document are already implemented by default.

Brief historical context

The topic of *symbol files* (=module interface files) has accompanied compiler development ever since the original *module* concept with *separate compilation* and type-checking *across* module boundaries (as opposed to *independent* compilation where no such checks are performed) has been introduced in the 1970s and adopted in languages such as Mesa, Ada, Modula-2 and Oberon.

A correct implementation of the *module* concept was by no means obvious initially. However, the concept has evolved and today, simple implementations exist covering all key requirements, e.g.,

1. *Re-export conditions*: Imported types may be *re-exported* and their *imports* may be hidden.
2. *Recursive data structures*: Pointer declarations may *forward reference* a record type.
3. *Cyclic references:* Record and pointer types may contain cyclic references among themselves.
4. *Module alias names*: A module can be imported under a different (alias) name.
5. *Hidden record fields:* Offsets of non-exported pointer fields are needed for garbage collection; offsets of non-exported procedure variable fields are needed for module reference checking.

A careful and detailed study of the evolution that led to today's status quo – which contains many useful lessons and is therefore well worth the effort – is far beyond the scope of this technical note. The reader is referred to the literature [1-13]. Here, a very rough sketch must suffice:

- Module concept introduced in 1972, early languages include Mesa, Modula and Ada [1].
- Modula-2 implementation on PDP-11 in 1979 already used *separate* compilation [2].
- Modula-2 implementation on Lilith in 1980 already used *separate* compilation [3].
- The first single-pass Modula-2 compiler in 1984 uses a *postorder* traversal [4, 5, 7].
- Oberon compilers in the 1990s used either a *postorder* or *preorder* traversal of the symbol table [8-12].
- The Oberon on ARM compiler in 2008 used a *fixup* technique for types in symbol files [13].
- The Project Oberon 2013 compiler uses *preorder* traversal and a *fixup* technique for types [14].

As with the underlying languages, all these re-implementations and refinements of the handling of import and export (and the associated symbol files) are characterized by a continuous *reduction* of complexity. In this note, we present yet another simplification by eliminating the so-called "fixup" technique for *types* during export and subsequent import, as well as some additional improvements.

Symbol files in ARM Oberon 2008 and in Project Oberon 2013

The Oberon system and compiler were re-implemented around 2013 on an FPGA development board. The compiler was derived from an earlier version of the Oberon compiler for the ARM processor. In the Project Oberon 2013 compiler, the same "fixup" technique to implement forward references *in* symbol files as in the ARM Oberon compiler is used. Quoting from the *Oberon on ARM* report [13]:

If a type is imported again and then discarded, it is mandatory that this occurs before a reference to it is established elsewhere. This implies that types must always be defined before they are referenced.

[1] *http://www.github.com/andreaspirklbauer/Oberon-module-imports*
[2] *http://www.github.com/andreaspirklbauer/Oberon-extended*

Fortunately, this requirement is fulfilled by the language and in particular by the one-pass strategy of the compiler. However, there is one exception, namely the possibility of forward referencing a record type in a pointer declaration, allowing for recursive data structures:

TYPE P = POINTER TO R;
R = RECORD x, y: P END

Hence, this case must be treated in an exceptional way, i.e. the definition of P must not cause the inclusion of the definition of R, but rather cause a forward reference in the symbol file. Such references must by fixed up when the pertinent record declaration had been read. This is the reason for the term {fix} in the syntax of (record) types. Furthermore, the recursive definition

TYPE P = POINTER TO RECORD x, y: P END

suggests that the test for re-import must occur before the type is established, i.e. that the type's name must precede the type's description in the symbol file, where the arrow marks the fixup:

TYP [#14 P form = PTR [^1]]
TYP [#15 R form = REC [^9] lev = 0 size = 8 {y [^14] off = 4 x [^14] off = 0}] → 14

Observations

The excerpt above notes that types must always be defined before they are referenced during compilation. At first sight, this might suggest the need for fixups to handle forward or cyclic references. An alternative approach, however – described in [9] and [12] and also implemented in Project Oberon 2013 – structures the symbol file so that each unique type reference always precedes its corresponding type description. This arrangement is possible even in the presence of forward or cyclic dependencies.

The principle is that the linear description of an enclosing object *P* may open a *bracket* to include the complete description of a component object *R* (such as a record referenced by a pointer type, a record field within a record, or a parameter of a procedure type). After the bracket is closed, the description of *P* resumes. If the description of *R* in turn refers back to *P*, the reference number of *P* is used, thereby resolving the cyclic dependency without fixups.

The example above of a forward pointer declaration is now linearized as follows, without any fixups:

TYP [#14 P form = PTR [#15 R form = REC [^9] lev = 0 size = 8 {y [^14] off = 4 x [^14] off = 0}]]

Here, the pointer declaration *P* obtains a reference number #14 which precedes its actual type description, while the record declaration *R* with reference number #15 is *embedded* in the linear description of *P*. This eliminates the need for fixups. *R* refers back to the description of *P* using reference number #14.

As a result of this approach, all type references within symbol files inherently take the form of *backward* references. This design choice also simplifies the process of reconstructing the symbol table data structure during import, and makes it straightforward to ensure that types are always *defined* (i.e. inserted into the symbol table) before they are *referenced* (see *ORB.OutType* and *ORB.InType*):

```
PROCEDURE OutType(VAR R: Files.Rider; t: Type);
  ...
BEGIN
  IF t.ref > 0 THEN (*type was already output*) Write(R, -t.ref)
  ELSE obj := t.typobj;
    IF obj # NIL THEN Write(R, Ref); t.ref := Ref; INC(Ref) ELSE Write(R, 0) END ;
    Write(R, t.form);
    IF t.form = Pointer THEN OutType(R, t.base)
    ELSIF t.form = Array THEN OutType(R, t.base); ...
    ELSIF t.form = Record THEN OutType(R, t.base); ...
    ELSIF t.form = Proc THEN OutType(R, t.base); ...
    END
  END ;
  ...
END OutType;
```

```
PROCEDURE InType(VAR R: Files.Rider; thismod: Object; VAR T: Type);
  ...
BEGIN Read(R, ref);
  IF ref < 0 THEN T := typtab[-ref]  (*type was already read*)
  ELSE NEW(t); T := t; typtab[ref] := t; ...
    Read(R, form); t.form := form;
    IF form = Pointer THEN InType(R, thismod, t.base); ...
    ELSIF form = Array THEN InType(R, thismod, t.base); ...
    ELSIF form = Record THEN InType(R, thismod, t.base); ...
    ELSIF form = Proc THEN InType(R, thismod, t.base); ...
    END
  END ;
  ...
END InType;
```

One can easily verify that in Project Oberon 2013 types are *always* already "fixed up" with the right value by slightly modifying procedure *ORP.Import* as follows:

```
WHILE k # 0 DO
  IF typtab[k].base # t THEN ORS.Mark("type not yet fixed up") END ;
  typtab[k].base := t; Read(R, k)
END
```

The error message "type not zet fixed up" will *never* be printed when importing a module. A more formal proof can of course easily be constructed. It rests on the observation that a type's *reference* number is written to the symbol file *before* the remaining type description and also before any other types or objects refer to this type.

Code that can be omitted in Project Oberon 2013

As a result of the approach described above, the following code (in red) in *ORB.Import* and *ORB.Export* can be omitted.

```
PROCEDURE Import*(VAR modid, modid1: ORS.Ident);
  ...
BEGIN
  ...
  Read(R, class);
  WHILE class # 0 DO
    NEW(obj); obj.class := class; Files.ReadString(R, obj.name);
    InType(R, thismod, obj.type); obj.lev := -thismod.lev;
    IF class = Typ THEN
      t := obj.type; t.typobj := obj; Read(R, k);
      (*fixup bases of previously declared pointer types*)
      WHILE k # 0 DO typtab[k].base := t; Read(R, k) END
    ELSE ...
      IF class = Const THEN ...
      ELSIF class = Var THEN ...
      END
    END
    obj.next := thismod.dsc; thismod.dsc := obj; Read(R, class)
  END ;
  ...
END Import;

PROCEDURE Export*(VAR modid: ORS.Ident; VAR newSF: BOOLEAN; VAR key: LONGINT);
  ...
BEGIN
  ...
  obj := topScope.next;
  WHILE obj # NIL DO
    IF obj.expo THEN Write(R, obj.class); Files.WriteString(R, obj.name);
      OutType(R, obj.type);
      IF obj.class = Typ THEN
        IF obj.type.form = Record THEN obj0 := topScope.next;
          (*check whether this is base of previously declared pointer types*)
          WHILE obj0 # obj DO
```

```
            IF (obj0.type.form = Pointer) & (obj0.type.base = obj.type)
              & (obj0.type.ref > 0) THEN Write(R, obj0.type.ref) END ;
            obj0 := obj0.next
          END
        END ;
        Write(R, 0)
      ELSIF obj.class = Const THEN ...
      ELSIF obj.class = Var THEN ...
      END
    END ;
    obj := obj.next
  END ;
  ...
END Export;
```

Handle type alias names among imported and re-imported modules correctly

In Project Oberon 2013, compilation of module *C* below leads to a compilation error.

```
MODULE M;
  TYPE A* = RECORD END ;
    B* = A;
END M.

MODULE M0;
  IMPORT M;                 (*import type M.A and type alias name M.B*)
  VAR a*: M.A;              (*re-export type M.A, but the type name M.B is (incorrectly) written in PO 2013*)
END M0.

MODULE C;
  IMPORT M0, M;             (*first re-import type M.A via M0 and then directly import type M.B from M*)
  VAR c: M.A;               (*compilation error in Project Oberon 2013 if explicit import of M were allowed*)
END C.
```

The first reason is that the explicit import of module *M* in module *C* is not allowed in Project Oberon 2013, as module *C* has already re-imported the type *M.A* via module *M0*, resulting in an *invalid import order* error.

But even if it were permitted, this scenario would trigger a compilation error during compilation of module *C* due to the unknown type *M.A* when processing the declaration of the global variable *c*. The issue arises from the import process in module *M0*, where the imported type alias *M.B* is correctly identified as the previously re-imported type *M.A*. However, the subsequent assignment *obj.type.typobj := obj* in *ORB.Import* incorrectly redirects the back-pointer for *M.B* to the newly imported type alias object *M.B* instead of leaving it as pointing to *M.A*. Consequently, when *M.A* is re-exported by module *M0*, it is misrepresented as *M.B* in the symbol file of *M0*, rendering the original type *M.A* inaccessible to clients like module *C*.

To solve this issue, it suffices to replace the following statement in *ORB.Import*:

```
t.typobj := obj
```

with:

```
IF t.typobj = NIL THEN t.typobj := obj END
```

which establishes the *typobj* back-pointer only if it doesn't exist yet. This ensures that an imported type alias name always points to the *original* imported type object, not another imported type alias object.

This is the same type of precaution as in *ORP.Declarations*, where type aliases declared in the module being compiled are initialized as follows:

```
IF tp.typobj = NIL THEN tp.typobj := obj END
```

which also makes sure that a type alias name declared in the module currently being compiled always points to the *original* type object (no matter whether the original type is imported from another module or declared in the module currently being compiled).

As this example shows, special care must be taken when handling cases where *imported* or *re-imported* types have type alias names associated with them.

Consider the following scenario:

```
MODULE M;
  TYPE A* = RECORD END ;        (*exported original type*)
    B = A;                      (*non-exported type alias name of an exported original type A*)
    C* = B;                     (*exported type alias name of (another alias of) an exported original type A*)
    D = RECORD END ;            (*non-exported original type, considered local to module M*)
    (*E* = D;*)                 (*export of E not allowed, because the original type D is not exported*)
END M.

MODULE M0;
  IMPORT M;
  TYPE F* = M.C;                (*type alias name of an imported type alias name M.C, resolves to M.A*)
  VAR c*: M.C;                  (*original type M.A re-exported, not the type alias name M.C*)
    f*: F;                      (*original type M.A re-exported, as well as the type alias name F*)
END M0.

MODULE C;
  IMPORT M0, M;                 (*original type M.A re-imported via M0 and explicitly imported from M*)
  VAR c*: M.C;                  (*type alias name M.C of an original type M.A directly imported from M*)
END C.
```

In our implementation, type alias names are handled as follows:

- A *type alias name* is simply an additional name for an existing type; it does not define a new type. A type may have multiple alias names, and if a type alias name refers to another type alias name, it ultimately resolves to the original type.

- If an alias of an imported type is exported or re-exported, the compiler always re-exports the original type as well. For example, exporting the type alias name *F* in module *M0* causes the original type *M.A* to be re-exported by *M0*, in addition to the type alias name *F*. This ensures that all references to *M.A*, whether accessed directly through *M* or indirectly through *M0*, refer to the same type object in the compiler's symbol table, thereby preserving type identity across modules.

- Both original types and type alias names can be directly imported from the modules in which they are defined. If a client module imports a type alias name and exports or re-exports a variable of that type, the re-export is expressed in terms of the original type, not the type alias name. For example, module *C* directly imports the type alias name *M.C* from *M*. When *C* exports the global variable *c*, the original type *M.A* is re-exported by *C*, not the alias *M.C*.

- Exported original types are written to a symbol file before any of their type alias names are written to the same symbol file. This guarantees that when a type such as *M.A* is first re-imported through an intermediate module *M0* and later explicitly imported by a client *C*, its first occurrence in the symbol file of *M* is identified as an original type. In this case, no new object node needs to be created in the compiler's symbol table (as it was already re-imported before through *M0*).

- A type alias name declared in the same module as its original type can only be exported if the original type is also exported. Allowing the export of a type alias name of a non-exported type would effectively expose a type not intended to leave the module and could alter semantics.

- As an alternative design, the export of type alias names could be disallowed entirely, restricting them to purely local use, similar to module alias names. In our design, this restriction has not been adopted.

In passing, we note the following general *export rules*, as realized in our implementation:

- Type alias names can only be exported if the *original* type is exported or is itself explicitly imported.
- Array types or variables of an array type can only be exported if the array *base* type is exported.
- A record field can only be exported if the type of the *field* and the type of the *record* are exported.

- Extended record types *can* be exported even if the record *base* type is not exported.
- Pointer types or pointer variables *can* be exported even if the pointer base type is not exported.
- Formal parameters and return types of exported procedures must be of exported data types.
- Anonymous variables cannot be exported.
- String constants can be exported as read-only (Extended Oberon only).

Allow re-imports to co-exist with module alias names and globally declared identifiers

In Project Oberon 2013, compilation of modules *M1* and *M2* below leads to a name conflict with the re-imported module name *M*:

```
MODULE M;
  TYPE T* = RECORD END ;
END M.

MODULE M0;
  IMPORT M;
  VAR t*: M.T;                    (*re-export type M.T*)
END M0.

MODULE M1;
  IMPORT M0;                      (*re-import type M.T*)
  VAR M: INTEGER;                 (*name conflict with globally declared identifier in Project Oberon 2013*)
END M1.

MODULE M2;
  IMPORT M := M0;                 (*name conflict with module alias name in Project Oberon 2013*)
END M2.
```

To solve this issue, we *hide* re-imported modules from the global namespace, allowing them to coexist with global identifiers and module alias names of explicitly imported modules. This is implemented by *skipping* over re-imports (identified as *~obj.rdo*) in various loops in module *ORB:*

```
PROCEDURE NewObj*(VAR obj: Object; id: ORS.Ident; class: INTEGER);
  VAR new, x: Object;
BEGIN x := topScope;
  WHILE (x.next # NIL) & ((x.next.name # id) OR (x.next.class = Mod) & ~x.next.rdo) DO
    x := x.next
  END ;
  ...

PROCEDURE thisObj*(): Object;
  VAR s, x: Object;
BEGIN s := topScope;
  REPEAT x := s.next;
    WHILE (x # NIL) & ((x.name # ORS.id) OR (x.class = Mod) & ~x.rdo) DO
      x := x.next
    END ;
    ...

PROCEDURE ThisModule(name, orgname: ORS.Ident; decl: BOOLEAN; key: LONGINT): Object;
  VAR mod: Module; obj, obj1: Object;
BEGIN obj1 := topScope;
  IF decl THEN obj := obj1.next;  (*search for alias, obj.class = Mod implicit*)
    WHILE (obj # NIL) & ((obj.name # name) OR ~obj.rdo) DO obj := obj.next END
    ...
```

If a re-imported module is later also explicitly imported, it is converted to an explicitly imported one in the symbol table, to ensure that its name *cannot* co-exist with module alias names or global identifiers.

Allow reusing the original module name if a module has been imported under an alias name

The Oberon language report defines aliased module imports as follows: *If the form "M := M1" is used in the import list, an exported object x declared within M1 is referenced in the importing module as M.x.*

Unfortunately, this definition allows for several different interpretations, for example:

Interpretation A:

- *It is module M1 that is imported, not M*
- *The module alias name M renames module M1 and the original name M1 can be reused*
- *A module can only have a single module alias name*

Interpretation B:

- *It is module M1 that is imported, not M*
- *A module alias name M is just an additional name for M1 and the original name M1 cannot be reused*
- *A module can have multiple module alias names*

Interpretation C:

- *It is module M that is imported, but the file with name M1 is read*

In our implementation, we have adopted interpretation A. It implies that the statement *IMPORT M := M1* imports module *M1* and associates the local name *M* with it, i.e. the identifier *M* is used as usual, but the file with name *M1* is read. The alias name *M* is substituted for the original name *M1* (which can be reused).

For example, the following scenarios are all legal:

```
MODULE A1; IMPORT M0 := M1, M1 := M2; END A1.
MODULE A2; IMPORT M0 := M1, M1 := M0; END A2.
MODULE A3; IMPORT M0 := M1, M2 := M0; END A3.
```

whereas the following scenario is illegal:

```
MODULE B1; IMPORT M1, A := M1, B := M1; END B1.
```

This is implemented by *not* checking the two combinations *obj.orgname # name* and *obj.name # orgname*, where *obj* denotes an existing module in the module list of the symbol table. Not checking the combination *obj.orgname = name* allows the second import *M1 := M2* with *name = M1* in module *A1*. Not checking the combination *obj.name = orgname* allows the second import *M1 := M0* with *orgname = M0* in module *A2*.

This leaves us with checking the *other* two combinations *obj.name # name* and *obj.orgname # orgname*:

```
PROCEDURE ThisModule(name, orgname: ORS.Ident; decl: BOOLEAN; key: LONGINT): Object;
  VAR mod: Module; obj, obj1: Object;
BEGIN obj1 := topScope;
  IF decl THEN  (*explicit import by declaration*)
    obj := obj1.next;  (*search for alias*)
    WHILE (obj # NIL) & ((obj.name # name) OR ~obj.rdo) DO obj := obj.next END
  ELSE obj := NIL
  END ;
  IF obj = NIL THEN obj1 := obj1.next;  (*search for module*)
    WHILE (obj # NIL) & (obj.orgname # orgname) DO obj1 := obj; obj := obj1.next END;
    IF obj = NIL THEN (*insert new module*) ...
    ELSE (*module already present*)
      IF decl THEN (*explicit import by declaration*)
        IF obj.rdo THEN ORS.Mark("mult def")
        ELSE obj.name := name; obj.rdo := TRUE  (*convert obj to explicit import*)
        END
      END
    END
  ELSE ORS.Mark("mult def")
  END ;
  RETURN obj
END ThisModule;
```

Propagate imported export numbers of type descriptor addresses to client modules

The Project Oberon 2013 implementation does not support type tests or type guards on types that are only re-imported via other modules, but are not explicitly imported. This doesn't pose an issue since only explicitly imported types can be referenced by name in client modules anyway.

But our implementation allows explicitly importing a module *M* even *after* types of *M* have previously been re-imported through another module, as outlined in the next section. In such cases, the previously re-imported module is simply *converted* to an explicitly imported one in the symbol table of the compiler. To enable type tests and type guards on types declared in such "converted" modules, we have replaced the following code in *ORB.OutType*:

```
ELSIF t.form = Record THEN ...
  IF obj # NIL THEN Files.WriteNum(R, obj.exno) ELSE Write(R, 0) END ;
  ...
```

with:

```
ELSIF t.form = Record THEN ...
  IF obj # NIL THEN
    IF t.mno > 0 THEN Files.WriteNum(R, t.len) ELSE Files.WriteNum(R, obj.exno) END
  ELSE Write(R, 0)
  END ;
```

This makes sure that *imported* export numbers of type descriptor addresses (stored in the field *t.len*) are re-exported to client modules, thereby enabling type tests and type guards on such types.

Allow an explicit import after previous re-imports of types of the same module

In the Oberon programming language, imported types can be re-exported and their original import may be hidden from the re-importing module during the *import process*. This means that a type *T* from one module *(M)* can be imported by another module *(M1)* and then re-exported to a third module *(M2)* without *M2* being aware of the original import from *M*. Two common approaches to implement this mechanism include:

1. *Self-contained symbol files*: This approach involves including imported types in symbol files, making the files self-contained and complete. This ensures that all required information is available in each symbol file, eliminating the need for recursive imports.

2. *Recursive imports*: In this approach, all required symbol files are imported recursively in full, to ensure that all necessary types are available for use. This method can result in more imports, but it is more transparent and easier to understand the dependencies between modules.

Project Oberon 2013 has chosen the first approach (self-contained symbol files). But it does not allow an explicit import of a module *M* after types of *M* have previously been re-imported via other modules. Our implementation removes this limitation. Consider the following scenario*:*

```
MODULE M;                          (*base module*)
  TYPE T0* = RECORD END ;
    T1* = RECORD END ;
    T2* = RECORD END ;
END M.

MODULE M0;                         (*intermediate module*)
  IMPORT M;                        (*import types M.T0, M.T1 and M.T2 from M directly*)
  VAR t0*: M.T0;                   (*re-export type M.T0 to clients of M0*)
END M0.

MODULE M1;                         (*intermediate module*)
  IMPORT M;                        (*import types M.T0, M.T1 and M.T2 from M directly*)
  VAR t0*: M.T0;                   (*re-export type M.T0 to clients of M1*)
    t1*: M.T1;                     (*re-export type M.T1 to clients of M1*)
END M1.
```

where the modules *M*, *M0* and *M1* are imported by the client modules *A*, *B* and *C* as follows:

```
MODULE A;                          (*allowed in Project Oberon 2013 and in Extended Oberon*)
  IMPORT M0,                       (*re-import type M.T0 via M0*)
    M1;                            (*re-import types M.T0 and M.T1 via M1*)
END A.

MODULE B;                          (*allowed in Project Oberon 2013 and in Extended Oberon*)
  IMPORT M,                        (*import types M.T0, M.T1 and M.T2 from M directly*)
    M0,                            (*re-import type M.T0 via M0*)
    M1;                            (*re-import types M.T0 and M.T1 via M1*)
END B.

MODULE C;                          (*not allowed in Project Oberon 2013, allowed in Extended Oberon*)
  IMPORT M0,                       (*re-import type M.T0 via M0*)
    M1,                            (*re-import types M.T0 and M.T1 via M1*)
    M;                             (*import types M.T0, M.T1 and M.T2 from M directly*)
END C.
```

A robust implementation must correctly handle *named* types in *every* possible scenario, including explicit imports and re-imports from multiple symbol files or any combination thereof, in any order. The principal requirement is to ensure that, regardless of how many times a particular type is encountered during the import process and irrespective of the order of the various (re-)imports of this type, it consistently maps to a *single* node in the symbol table of the compiler. Failure to achieve this can lead to incompatibilites during *type checking*. Recall that in typical implementations, type equality is determined by comparing pointers referencing type descriptors in the symbol table. This is made possible by the Oberon language definition, which specifies equivalence of types on the basis of names rather than structure.

In the above example, the type *M.T0* is imported and re-imported across the various modules as follows:

- During compilation of module *A*, the type *M.T0* is *re-imported* twice: first, when module *M0* is imported and second, when module *M1* is imported. During the second re-import via module *M1*, the type *M.T0* is discovered within the object list of module *M,* because it has already been re-imported via module *M0* before. In this example, the symbol file of the declaring module *M* is never loaded.

- During compilation of module *B*, the type *M.T0* is first *explicitly* imported when the symbol file of module *M* is loaded and then *re-imported* twice: first, when module *M0* is imported and second, when module *M1* is imported. During both re-imports, the type *M.T0* is discovered within the object list of module *M*, because the symbol file of the declaring module *M* has been loaded before.

- During compilation of module *C*, the type *M.T0* is first *re-imported* twice via modules *M0* and *M1* and subsequently *explicitly* imported from module *M*. During the second re-import via module *M1* and the explicit import from module *M*, the type *M.T0* is discovered within the object list of module *M*, because it has already been re-imported via module *M0* before.

The Project Oberon 2013 implementation already has a built-in mechanism to identify instances where a type to be re-imported is already present in the compiler's symbol table. This may be the case because the symbol file of the module defining the type has already been loaded, or because the type has already been read when loading other symbol files. The implementation is straightforward and involves a simple search for the type's name within the object list of the declaring module *M* (see *ORB.InType*):

```
IF modname[0] # 0X THEN (*re-import*)
  Files.ReadInt(R, key); Files.ReadString(R, name);
  mod := ThisModule(modname, modname, FALSE, key);
  obj := mod.dsc;  (*search type*)
  WHILE (obj # NIL) & (obj.name # name) DO obj := obj.next END ;
  IF obj # NIL THEN T := obj.type  (*type object found in object list of mod*)
  ELSE (*insert new type object in object list of mod*)
    NEW(obj); obj.name := name; obj.class := Typ; obj.next := mod.dsc;
    mod.dsc := obj; obj.type := t; t.mno := mod.lev; t.typobj := obj; T:= t
  END ;
  typtab[ref] := T
END
```

This mechanism covers the case where a type is first explicitly imported and then re-imported via other modules (once or several times), as well as the case where a type is re-imported solely via other modules (once or several times) without its declaring module ever being explicitly imported.

But it does not cover the case where a type is first re-imported via other modules (once or several times) and then explicitly imported from its declaring module, as in module *C* above.

Instead, the Project Oberon 2013 implementation enforces a restriction: It prevents explicit imports of modules from which individual types have previously been re-imported via other modules.

If we want to also allow *explicit* imports *after* prior re-imports of the same type, we could of course employ the same technique that is already used for handling *re-imports*.

```
IF modname[0] # 0X THEN (*re-import*) ...
ELSE (*explicit import*)
  Search the type in the object list of the currently imported module. If the
  type is found, map it to the type node of this (previously re-imported) type.
END
```

But this approach would involve searching for each explicitly imported type within the object list of the currently imported module *M*. Furthermore, it would require additional modifications in module *ORB* to ensure that the type's name can be accessed in *ORB.InType*.

An alternative approach consists of propagating the original reference number of a re-exported type *t*, denoted as *t.orgref*, across the module hierarchy and by using this reference number to initialize the compiler's *type translation table*[3] for the module prior to its explicit import. This eliminates the need to search for each explicitly imported type within the compiler's symbol table.

The original reference number of a type *t* is propagated as follows:

- If a type *t* = *M.T* declared in module *M* is directly imported by a module *M0*, the field *t.orgref* is set to the reference number of "t in M" (read from the symbol file of the declaring module *M* itself). This effectively marks the beginning of the chain of re-exports and subsequent re-imports of this type[4].

- If *t* is re-exported by *M0*, a reference number for the type "M.T in M0" *(t.ref)* is assigned to the type and written to the symbol file of *M0*, together with its reference number in its declaring module *M* (*t.orgref*).

- If *t* is subsequently *re-imported* by a client via module *M0*, the field *t.orgref* is again set to the reference number of *t* in its *declaring* module *M* (read from the symbol file of the re-exporting module *M0*).

Note that the field *t.orgref* is only written to symbol files if the type *t* is *re-exported*. Otherwise it is implicit. Since re-exports are rare, this has a negligible effect on the overall length of symbol files.

With this preparation, our implementation approach can be summarized as follows:

- When a module *M* exports a type *t* to an intermediate module *M0* and a client module *C* subsequently re-imports this type via module *M0*, a module object for its declaring module *M* and a type object for the re-imported type in the object list of module *M* is inserted into the compiler's symbol table during compilation of *C*, together with its original reference number (*t.orgref*) in its declaring module *M*.

- If the same client *C* later also *explicitly* imports module *M*, we start by initializing the compiler's type translation table for module *M* with all types of *M* that have previously been re-imported via other modules, using the original reference numbers in their declaring module *M* as the index (this is why they are propagated). In the above example, these are the types *M.T0* and *M.T1*.

- For convenience, we also *mark* each previously re-imported type (e.g., by temporarily inverting the sign of its module number) during this initialization phase. This will allow us to easily detect, whether a type read from the symbol file of *M* has previously been re-imported via *other* modules.

[3] *The compiler's type translation table (typtab) for a module M is a table containing references to all types of M that already exist in the object list of M.*
[4] *For the re-export mechanism to function, the symbol file of M must be read entirely at least once before initiating the chain of re-exports and re-imports of individual types of M.*

- If module *C* then reads a type *t* from the symbol file of module *M* directly, there are two cases:

  Case A: If the type *t has* previously been re-imported via other modules, we reuse the already existing type, while continuing to read the type information of *t* from the symbol file of *M*. In the above example, this is the case when module *C* reads the types *M.T0* and *M.T1* from the symbol file of *M*. Note that only *named* types can ever be re-exported. Since named types are written to symbol files *before* variables and procedures that might refer to them, we know that the object *class* must be *Typ* in this case and therefore no additional data needs to be read from the symbol file of *M*.

  Case B: If the type *t* has *not* previously been re-imported via other modules, we create and insert a new type object for *t* into the object list of module *M*. In the above example, this is the case when module *C* reads the type *M.T2* from the symbol file of *M*. This is the regular (and frequent) case.

- Now that we allow explicit imports after re-importing types from the same module, we must ensure that named types, which are re-exported and re-imported across multiple modules, cannot be referenced by name in modules that import them only indirectly. One possible approach would have been to check in the importing module whether these types were originally exported in their declaring module. However, we chose a different approach. Our implementation simply *requires* that types are exported whenever they are used in the declarations of exported variables or types. This rule also applies to record fields and array base types.

The following code excerpts show a possible implementation of this scheme:

ORB.Import:

```
thismod := ThisModule(modid, modid1, TRUE, key);
FOR i := Record+1 TO maxTypTab-1 DO typtab[i] := NIL END ;
obj := thismod.dsc;  (*initialize typtab with already re-imported types*)
WHILE obj # NIL DO
  typtab[obj.type.orgref] := obj.type;  (*initialize typtab*)
  obj.type.mno := -obj.type.mno;  (*mark type as re-imported*)
  obj := obj.next
END ;
...
Read(R, class);
WHILE class # 0 DO
  Files.ReadString(R, name); InType(R, thismod, t);
  IF t.mno < 0 THEN t.mno := -t.mno  (*type already re-imported via other modules*)
  ELSE NEW(obj);  (*insert new type object in object list of thismod*)
    obj.class := class; obj.name := name; obj.type := t; obj.lev := -thismod.lev;
    IF class = Const THEN ...
    ELSIF class = Var THEN ...
    ELSIF t.typobj = NIL THEN t.typobj := obj
    END ;
    obj.next := thismod.dsc; thismod.dsc := obj
  END ;
  Read(R, class)
END
```

ORB.InType:

```
Files.ReadString(R, modname);
IF modname[0] # 0X THEN (*re-import*) ...
  Files.ReadInt(R, key); Files.ReadString(R, name); Read(R, orgref);
  mod := ThisModule(modname, modname, FALSE, key);
  obj := mod.dsc;  (*search type*)
  WHILE (obj # NIL) & (obj.name # name) DO obj := obj.next END ;
  IF obj # NIL THEN T := obj.type  (*type object found in object list of mod*)
  ELSE (*insert new type object in object list of mod*)
    NEW(obj); obj.name := name; obj.class := Typ; obj.next := mod.dsc; mod.dsc := obj;
    obj.type := t; t.mno := mod.lev; t.typobj := obj; t.orgref := orgref
  END
ELSE (*explicit import*)
  IF typtab[ref] # NIL THEN T := typtab[ref] END  (*reuse already re-imported type*)
END
```

In passing, we note that Project Oberon 2013 allows a maximum of 15 modules to be imported by any single module. This typically doesn't pose any issues, as it aligns with the good programming practice of structuring the module hierarchy in a way that only a small number of modules are imported.

However, this upper limit also includes modules from which types are (only indirectly) *re-imported*. These modules don't necessarily have to be explicitly listed in the import statement; their imports can remain hidden. Therefore, in deep module hierarchies, there may arise a desire to lift this restriction. To address this need, our implementation increases the maximum number of modules that can be directly or indirectly imported from 15 to 63, providing greater flexibility in managing complex module structures.

Write the module anchor of re-exported types before the type description to the symbol file

When implementing the re-export mechanism through *self-contained symbol files*, it is essential to include in the type description a reference to the module in which a re-exported type was originally defined. This reference, known as the *module anchor*, typically includes the module name and key of the respective module. When combined with the type's name, it forms a unique *identifier* for the re-exported type. Note that in our implementation, this module anchor is only written for re-exported types. For types declared in (and exported by) the currently compiled module, the reference to the declaring module is implicit.

In contrast to Project Oberon 2013, our implementation writes this unique identifier of a re-exported type immediately *after* the type's reference number, but *before* the actual type description to the symbol file of the re-exporting module. This approach corresponds to postulate #5 in [5]. It guarantees that no other types can appear between the type's *reference* number and its *identification*, thereby ensuring that the code also works in the presence of cyclic references *among re-imported types.*

Recall that a type may refer to itself, as illustrated in the following example:

```
MODULE M;
  TYPE P1* = POINTER TO R1;
    P2* = POINTER TO R2;
    P3* = POINTER TO R3;
    R1* = RECORD p2*: P2 END ;
    R2* = RECORD p1*: P1 END ;                (*cyclic reference through record fields*)
    R3* = RECORD (R1) p3*: P3 END ;           (*cyclic reference through type extensions*)
END M.
```

Consider the case where the types defined in module *M* are re-imported by a client via an intermediate module *M0*. In this situation, procedure *ORB.InType* is recursively called for the *re-imported* types *P1, R1, P2, R2* and *R1* (in this order) and the *last* call to *ORB.InType(R, thismod, t.base)*, made when reading the type *R2*, sets *t.base* to *R1* via its variable parameter *T*.

If the module anchor of the declaring module *M* were stored in the symbol file of the re-exporting module *M0 after* the type description of *R1* (as in Project Oberon 2013), the code to read this module anchor would also be executed *after* the recursive calls to *ORB.InType* for the re-imported types *P1, R1, P2, R2* and *R1:*

```
PROCEDURE InType(VAR R: Files.Rider; thismod: Object; VAR T: Type);
  ...
BEGIN Read(R, ref);
  IF ref < 0 THEN T := typtab[-ref]
  ELSE NEW(t); T := t;
    ...
    InType(R, thismod, t.base);               (*recursive call to InType, changes t.base*)
    ...
    (*code to read the module anchor*)
    Files.ReadString(R, modname);
    IF modname[0] # 0X THEN (*re-import*)
      ...
      T := obj.type;                          (*changes t.base one level up in the recursion*)
      ...
   END
  END
END InType;
```

But the code handling re-imports may change the field *t.base* one level up in the recursion via the variable parameter *T* to an entry in the compiler's type translation table or an existing entry in the object list of the declaring module *M*.

While this doesn't actually pose an issue – because the code will eventually discard the subtree rooted in *t* if it detects that this type has previously been read from another symbol file – our preference is to establish the data structure for re-imported module and type objects in the symbol table of the compiler *before* entering any recursion. This prevents the type *T* from temporarily holding an incorrect value and builds the data structure for re-imported modules in the compiler's symbol table *before* any of their types are inserted into their respective object lists.

In our implementation, we have therefore decided to move the code to read and write the module anchor of re-imported and re-exported types to the beginning of *ORB.InType* and *ORB.OutType*, as shown below:

```
PROCEDURE InType(VAR R: Files.Rider; thismod: Object; VAR T: Type);
  ...
BEGIN Read(R, ref);
  IF ref < 0 THEN T := typtab[-ref]  (*already read*)
  ELSE NEW(t); T := t; t.mno := thismod.lev; t.orgref := ref;
    IF ref > 0 THEN  (*named type*)
      (*code to read the module anchor*)
      Files.ReadString(R, modname);
      IF modname[0] #  0X THEN  (*re-import*)
        Files.ReadInt(R, key); Files.ReadString(R, name); Read(R, orgref);
        mod := ThisModule(modname, modname, FALSE, key);
        obj := mod.dsc;  (*search type*)
        WHILE (obj # NIL) & (obj.name # name) DO obj := obj.next END ;
        IF obj # NIL THEN T := obj.type  (*type object found in object list of mod*)
        ELSE NEW(obj); (*insert new type object in object list of mod*)
          obj.name := name; obj.class := Typ; obj.next := mod.dsc; mod.dsc := obj;
          obj.type := t; t.mno := mod.lev; t.typobj := obj; t.orgref := orgref
        END
      ELSIF typtab[ref] # NIL THEN T := typtab[ref]  (*already re-imported*)
      END ;
      typtab[ref] := T
    END ;
    Read(R, form); t.form := form;
    IF form = Pointer THEN InType(R, thismod, t.base); ...
    ELSIF form = Array THEN InType(R, thismod, t.base); ...
    ELSIF form = Record THEN InType(R, thismod, t.base); ...
    ELSIF ...
    END
  END
END InType;

PROCEDURE OutType(VAR R: Files.Rider; t: Type);
  ...
BEGIN
  IF t.ref > 0 THEN (*type was already output*) Write(R, -t.ref)
  ELSE obj := t.typobj;
    IF obj # NIL THEN  (*named type*)
      Write(R, Ref); t.ref := Ref; INC(Ref);
      IF t.mno > 0 THEN  (*re-export*)
        (*code to write the module anchor*)
        mod := topScope.next;
        WHILE (mod # NIL) & (mod.lev # t.mno) DO mod := mod.next END ;
        IF mod # NIL THEN
          Files.WriteString(R, mod(Module).orgname); Files.WriteInt(R, mod.val);
          Files.WriteString(R, obj.name); Write(R, t.orgref)
        ELSE ORS.Mark("re-export not found"); Write(R, 0)
        END
      ELSE Write(R, 0)
      END
    ELSE (*anonymous*) Write(R, 0)
    END ;
```

```
    Write(R, t.form);
    IF t.form = Pointer THEN ...
    ELSIF t.form = Array THEN ...
    ELSIF t.form = Record THEN ...
    ELSIF ...
    END
  END
END OutType;
```

* * *

References

1. Parnas D.L. *On the Criteria To Be Used in Decomposing Systems into Modules*. Comm ACM 15, 12 (December 1972)
2. Wirth N. *MODULA-2*. Computersysteme ETH Zürich, Technical Report No. 36 (March 1980) (ch. 15 describes the use of an implementation of Modula-2 on a DEC PDP-11 computer)
3. Geissmann L. *Separate Compilation in Modula-2 and the Structure of the Modula-2 Compiler on the Personal Computer Lilith*, ETH Zürich Dissertation No. 7286 (1983)
4. Wirth N. *A Fast and Compact Compiler for Modula–2*. Computersysteme ETH Zürich, Technical Report No. 64 (July 1985)
5. Gutknecht J. *Compilation of Data Structures: A New Approach to Efficient Modula–2 Symbol Files*. Computersysteme ETH Zürich, Technical Report No. 64 (July 1985)
6. Rechenberg, Mössenböck. *An Algorithm for the Linear Storage of Dynamic Data Structures*. Internal Paper, University of Linz (1986)
7. Gutknecht J. *Variations on the Role of Module Interfaces*. Structured Programming 10, 1, 40-46 (1989)
8. J. Templ. *Sparc–Oberon. User's Guide and Implementation*. Computersysteme ETH Zürich, Technical Report No. 133 (June 1990).
9. Griesemer R. *On the Linearization of Graphs and Writing Symbol Files.* Computersysteme ETH Zürich, Technical Report No. 156a (1991)
10. Pfister, Heeb, Templ. *Oberon Technical Notes.* Computersysteme ETH Zürich, Technical Report No. 156b (1991)
11. Franz M. *The Case for Universal Symbol Files.* Structured Programming 14: 136-147 (1993)
12. Crelier R. *Separate Compilation and Module Extension*. ETH Zürich Dissertation No. 10650 (1994).
13. Wirth N. *An Oberon Compiler for the ARM Processor*. Technical note (December 2007, April 2008), www.inf.ethz.ch/personal/wirth
14. Wirth N., Gutknecht J. *Project Oberon 2013 Edition*, www.inf.ethz.ch/personal/wirth